

UNIVERSITÀ DEGLI STUDI DI TRIESTE

Esame di Programmazione (160IN)

- **Titolo del progetto:** Survival Game – Gioco di Ruolo a Turni da Terminale
- **Realizzato da:** Daniele Deiana (IN05A00032)

Descrizione generale del programma

Survival Game è un'applicazione Java single-player con interfaccia da terminale. Il gioco simula un percorso di sopravvivenza post-apocalittico articolato su 10 turni (tappe narrative), con partenza da Edimburgo e arrivo ad Atene.

Il programma offre due modalità di avvio:

- **Modalità Classica:** Avvio dal primo turno per percorrere l'intera esperienza di gioco.
- **Modalità Test:** Selezione libera del turno di partenza (1-11) per consentire un collaudo immediato e semplice delle condizioni di vittoria/sconfitta.
L'undicesimo turno corrisponde alla vittoria.

Ogni turno comprende fino a 3 avanzamenti. Ad ogni sotto-azione l'utente può scegliere se curarsi, visualizzare l'inventario o avanzare; solo la scelta di avanzare fa progredire il contatore interno fino al cambio di turno.

Descrizione delle classi

- `Inventario.java`: Gestisce le risorse trovate dal giocatore.
Si tratta di un array di stringhe a 5 slot (es. bende, munizioni).
I suoi metodi contengono gli algoritmi necessari a manipolare l'inventario.
`aggiungi(String):void`, `rimuovi(String):void`, `mostra():void`,
`contiene(String):boolean`.
- `Giocatore.java`: Rappresenta il giocatore con le sue statistiche.
Ha come attributi i punti vita (hp), la salute massima (hpMax), un inventario (oggetto `Inventario`) e lo stato ferito (boolean).
I metodi `subisciDanno` e `cura` gestiscono l'aumento e la diminuzione degli hp del giocatore; sono scritti in **overloading** per accettare parametri differenti o valori predefiniti.

Sono inoltre presenti alcuni metodi per manipolare l'inventario:

`mostraInventario`, `aggiungiAllInventario`, `rimuoviDallInventario` e `inventarioContiene`. Invocano i metodi pubblici della classe `Inventario` rispettando il principio dell'**incapsulamento**.

Sono infine presenti un metodo `mostraStato():void`, dei getter per gli hp per verificare se il giocatore è ferito e un setter per quest'ultima condizione.

- `Evento.java`: Classe ereditaria di base per la gestione degli eventi nel gioco. Importa `java.util.Random` e presenta un attributo di tipo `protected` della classe `Random`, accessibile direttamente dalle sottoclassi. Il metodo `esegui(Giocatore):void` è destinato all'override polimorfico.
- `Rifugio.java`: Sottoclasse di `Evento`. Effettua l'override del metodo `esegui(Giocatore)`. Se il giocatore è ferito ne ripristina lo stato di salute; altrimenti ne aumenta gli hp. Assegna infine un oggetto casuale (benda o munizioni) all'inventario.
- `Conflitto.java`: Sottoclasse di `Evento`. Effettua l'**override** del metodo `esegui(Giocatore)`. Se l'inventario contiene munizioni, il danno subito dallo scontro si riduce. Calcola il danno in modo casuale entro range definiti e presenta una probabilità del 30% di infliggere lo stato ferito.
- `GestoreFile.java`: classe dedicata alle operazioni di lettura e scrittura dei file.
- `Main.java`: Punto di ingresso dell'applicazione; contiene il metodo `main`.
- `Game.java`: il vero e proprio motore del gioco. Gestisce la logica del gioco (mappata nel diagramma di flusso in allegato) e manipola `Giocatore` e `GestoreFile` tramite incapsulamento.
 - *Costruttore*: Inizializza `GestoreFile`, `Random` e `Scanner`.
 - `start(): void`: Stampa l'incipit e gestisce il menu principale (Riepilogo, Nuova Partita, Modalità Test, Uscita).
 - `nuovaPartita(int): void`: Regola il ciclo dei 10 turni e le condizioni di fine partita ($hp \leq 0$ oppure completamento del decimo turno).

- `eseguiTurno(): void`: Gestisce il sotto-ciclo dei 3 avanzamenti per turno.
- `gestisciCura(): void`: Verifica la presenza di bende e applica la cura.
- `gestisciAvanzamento():void`: Istanza dinamicamente `Rifugio` o `Conflitto` tramite `generaEventoCasuale():Evento` e ne invoca il metodo polimorfico `esegui`.
- `dots():void`: Metodo d'utilità estetica per la formattazione dell'output.

Lettura e scrittura – GestoreFile

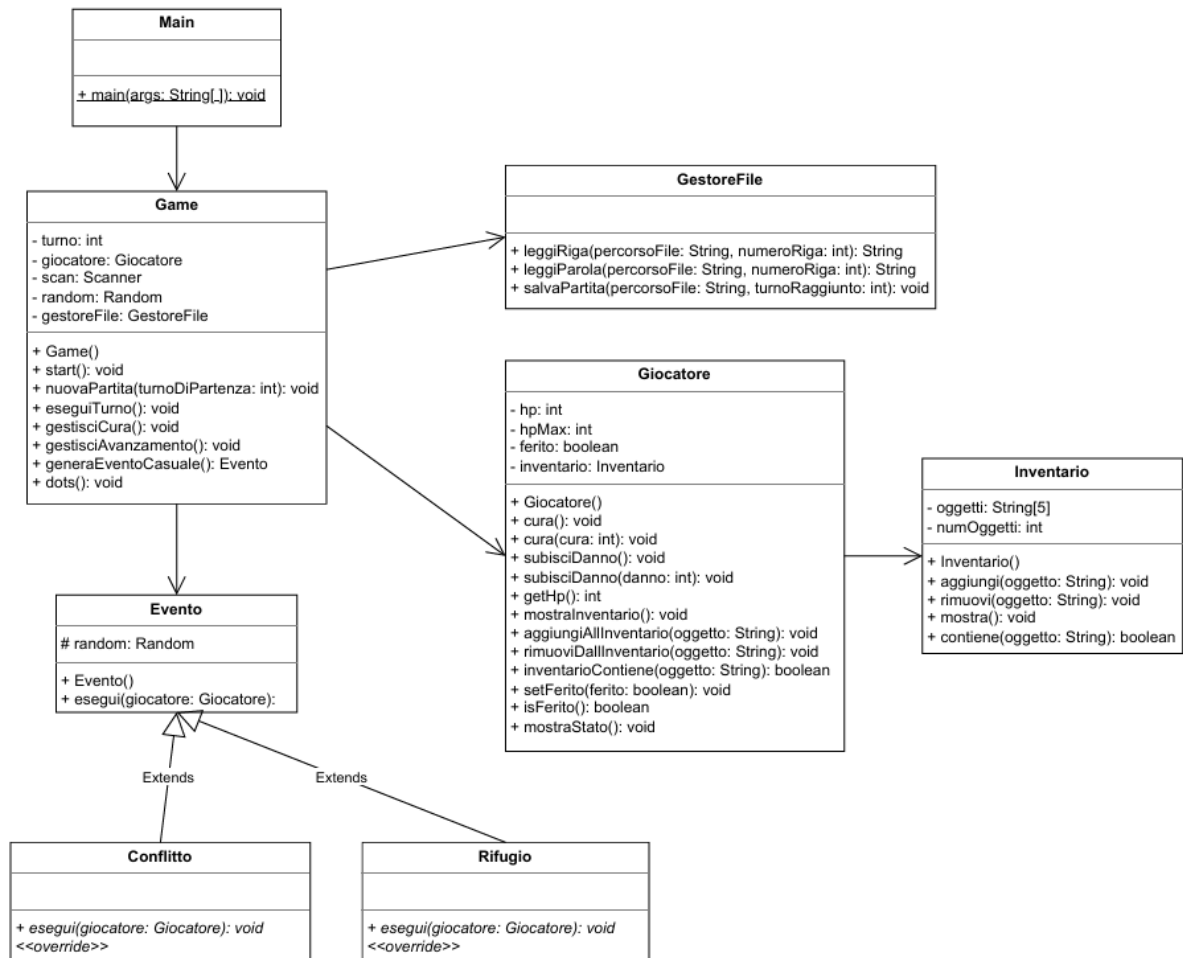
- **Lettura** (`leggiRiga`, `leggiParola`): Carica i testi di intro, vittoria e sconfitta dal file `inizioEfine.txt`, e le descrizioni narrative dei 10 turni dal file `tappe.txt`.
- **Scrittura** (`salvaPartita`): Registra il turno e la tappa raggiunti all'atto di vittoria o sconfitta nel file `riepilogo.txt`, consentendone la consultazione dal menu iniziale.

Gestione delle Eccezioni

Il programma gestisce le seguenti eccezioni nelle classi *Game.java* e *Main.java*, mentre le rilancia nella classe di utilità *GestoreFile.java*:

- **Input non validi** (`InputMismatchException`): Le operazioni di lettura da tastiera (`scan.nextInt()`) sono racchiuse in blocchi `try-catch`. In caso di inserimento di caratteri alfabetici o non validi, l'eccezione viene intercettata, il buffer dello `Scanner` viene ripulito tramite `scan.nextLine()` e l'utente viene reindirizzato al menu senza causare l'arresto anomalo del programma.
- **Eccezioni I/O** (`IOException`): Le operazioni di lettura e scrittura su disco intercettano eventuali errori di accesso o assenza dei file di testo richiesti. In caso di errore, il sistema mostra un messaggio informativo a schermo e prosegue l'esecuzione in modo controllato.
- **NoSuchElementException**: in caso di arresto del processo con `CTRL+C` il programma mostra un semplice e pulito messaggio di uscita senza indicare i processi interrotti
- **InterruptedException**: può avvenire con la funzione `Thread.sleep(int)`. Viene gestita nel metodo `dots` che utilizza tale funzione.

Diagramma UML delle classi



Nota: siccome la foto risulta essere sgranata aggiungo la tavola come allegato della mail.

